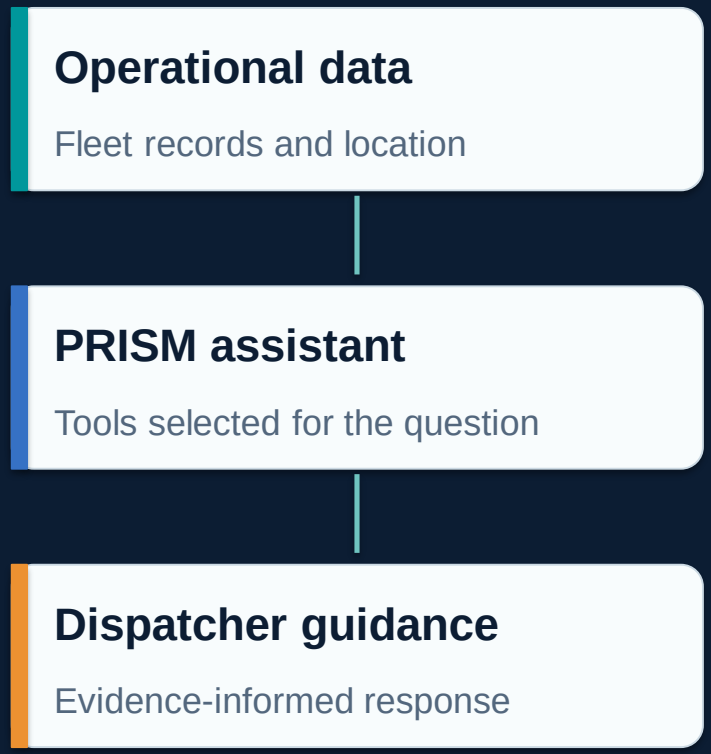
PRISM

Logistics operations assistant

A project-specific business overview

Fleet telemetry / Corridor conditions / Operating procedures



Built from the PRISM codebase and supplied operating procedure

Operations questions cross data boundaries

A dispatcher may need several sources to answer one shipment question.

Fleet record

What does the telemetry say?

Conditions

What is happening around the route?

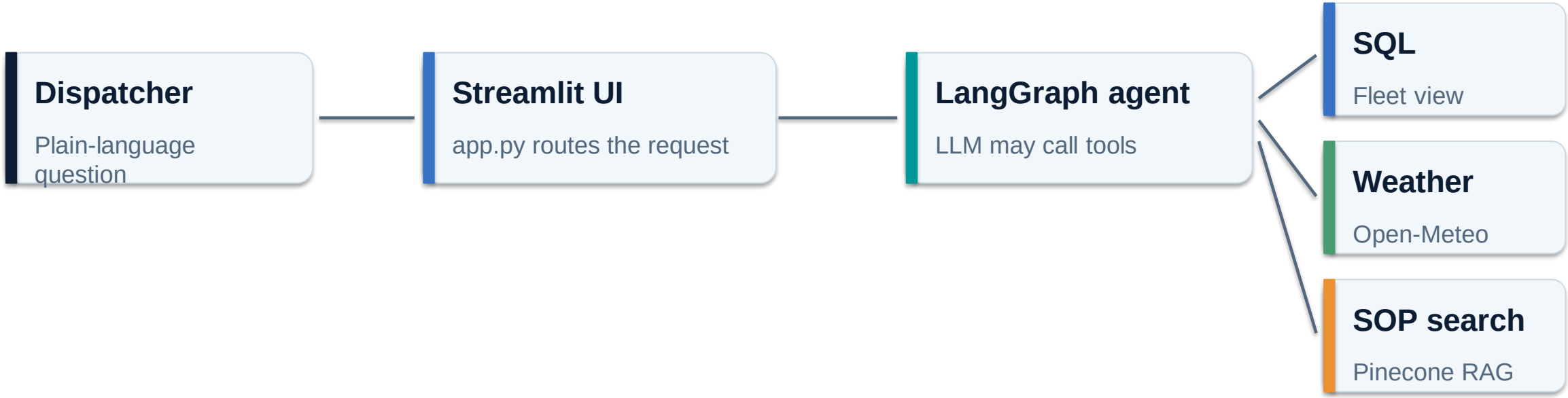
Procedure

Which operating rule applies?

PRISM puts these checks behind one conversational interface. The dispatcher still reviews the recommendation and makes the operational decision.

CURRENT APPLICATION FLOW

One assistant, three evidence tools



Tool results return to the agent, which writes a user-facing answer.

The model does not directly connect to a database. Python tools make the service calls.

TELEMETRY PATH

Fleet data flows through a restricted SQL view



inject_data.py loads mapped CSV fields into the raw table using replace mode.

security_and_view.sql defines selected readable columns and grants the AI account SELECT on the view. MySQL permissions provide the real access boundary.

Scope note: the current view selects the source table rows; its name does not itself filter records to only currently active shipments.

SOP search retrieves relevant procedure text

1	**Policy files**	Markdown, text, PDF, CSV, or XLSX
2	**Chunk and embed**	Local Hugging Face model
3	**Pinecone index**	384-dimensional vectors plus text metadata
4	**Question search**	Embed query and retrieve top 2 chunks
5	**Agent response**	Use retrieved clauses as supporting context

SOP rule example: fresh perishables above 4.0°C trigger a breach; the SOP directs driver contact and cooling restart, with diversion when ETA delay exceeds one hour.

The response is assembled during the request

- The user submits a question in the Dispatch Assistant.
- The app adds the system instructions and conversation thread context.
- The configured Groq or Gemini model can request one or more registered tools.
- ToolNode executes SQL, weather, or SOP retrieval and returns results to the model.
- The model formats its final response; Streamlit displays it and redacts detected secrets.

The LLM chooses tools at runtime. The system prompt asks for a sequence, but it does not force every integration to run on every question.

APPLICATION SURFACE

Four screens support exploration and use

Overview

Explains PRISM and its available sources.

Data explorer

Summarizes the expected local CSV, including risk, temperature, delay and locations.

Tool checks

Runs MySQL, weather and SOP integrations separately.

Dispatch assistant

Lets a user ask an operational question and receive a tool-supported response.

The explorer reads its CSV independently. It is a local historical-data view, not a live fleet feed.

Access controls exist, while some governance is not wired in

- The SQL tool accepts queries beginning with SELECT and reads through VW_ACTIVE_FLEET.
- The database account is intended to read the view and has separate INSERT/SELECT grants on AgentAuditLog.
- The audit table and grants exist, but the current application code does not write audit events.
- Weather comes from Open-Meteo; the displayed disruption score is a simple wind-speed rule, not a traffic feed.
- No Dockerfile or Compose file is present in this repository snapshot. A locally running MySQL service may still be an external dependency.

PROJECT POSITION

PRISM is a working prototype with a clear path to hardening

Implemented in this codebase

- Streamlit pages and conversational workflow
- MySQL fleet query tool and restricted-view SQL
- Pinecone SOP retrieval with local embeddings
- Open-Meteo weather lookup and response synthesis

Before production use

- Connect a verified, current operational data feed
- Wire audit writes and define retention/access controls
- Replace hard-coded SQL credentials; narrow database host access
- Validate weather/risk logic with operational owners

PRISM supports dispatcher review. It does not execute dispatch actions.